

AMAZON FRESH ANALYTICS

Domain:

E-Commerce and Retail Analytics

Tools Used:

MySQL Workbench | SQL | DDL | DML | TCL

Prepared by:

Ajaykumar.RP

June 2026

Table of Contents

Table of Contents	2
1. Project Overview.....	4
1.1 Problem Statement.....	4
1.2 Project Goals.....	4
1.3 Business Use Cases	4
1.4 Technical Stack.....	4
2. Data Modeling & ER Diagram	5
2.1 Entity-Relationship Overview (Task 1 & 2).....	5
2.2 Key Relationships.....	5
2.3 ER Diagram Description	5
3. Database Definition Language (DDL).....	6
3.1 Database Setup.....	6
3.2 Table Creation with Constraints (Task 4).....	6
4. Basic SELECT Queries (Task 3).....	7
4.1 Retrieve Customers from a Specific City	7
4.2 Fetch Products by Category	7
5. Data Manipulation Language (DML)	8
5.1 INSERT — Adding New Products (Task 5).....	8
5.2 UPDATE — Modifying Stock Quantity (Task 6)	8
5.3 DELETE — Removing a Supplier by City (Task 7)	8
6. SQL Constraints (Task 8).....	9
6.1 CHECK Constraint on Reviews.Rating	9
6.2 DEFAULT Constraint on Customers.PrimeMember	9
7. Clauses and Aggregations (Task 9)	9
7.1 WHERE Clause — Orders After January 1, 2024.....	9
7.2 HAVING Clause — Products with Average Rating > 4	9
7.3 GROUP BY & ORDER BY — Rank Products by Total Sales.....	10
8. Identifying High-Value Customers (Task 10).....	10
8.1 Total Spending Per Customer with Rank	10
8.2 Filter Customers Spending More Than ₹5,000	10
9. Complex Aggregations and Joins (Task 11).....	Error! Bookmark not defined.
9.1 Total Revenue Per Order (Orders JOIN Order_Details).....	11
9.2 Most Active Customers in 2025	11
9.3 Supplier with the Most Products in Stock.....	11
10. Normalization to 3NF (Task 12)	12
10.1 Problem with the Original Design.....	12

10.2 Solution — Separate Lookup Tables	12
10.3 Migration — Populate & Link	12
11. Subqueries and Nested Queries (Task 13)	12
11.1 Top 3 Products by Sales Revenue	13
11.2 Customers Who Have Not Placed Any Orders	13
12. Real-World Actionable Insights (Task 14)	13
12.1 Cities with the Highest Concentration of Prime Members	13
12.2 Top 3 Most Frequently Ordered Categories.....	13
13. ACID Transaction — TCL Commands (Bonus)	14
13.1 Transaction Implementation	14
13.2 ACID Properties Applied.....	14
14. Project Summary and Outcomes	15
14.1 What Was Accomplished.....	15
14.2 Key Learnings	16
14.3 Technical Tags.....	16

1. Project Overview

1.1 Problem Statement

Amazon Fresh is a large-scale e-commerce platform that specializes in groceries and daily essentials. As the platform grows, managing vast volumes of data — including customer profiles, product inventories, supplier information, orders, and customer reviews — becomes increasingly complex. The objective of this project is to design a fully normalized relational database for Amazon Fresh, perform meaningful SQL-based analysis, and generate actionable business insights to support strategic decision-making.

1.2 Project Goals

- Design an efficient, normalized relational database schema
- Implement DDL (Data Definition Language) to define tables and constraints
- Perform DML (Data Manipulation Language) operations: INSERT, UPDATE, DELETE
- Write advanced SQL queries using JOINS, subqueries, and aggregations
- Apply ACID transaction principles using TCL commands
- Derive actionable business insights from customer, product, and supplier data

1.3 Business Use Cases

Use Case	Description
Customer Insights	Analyze purchasing patterns to identify top customers and cities for targeted marketing
Product Management	Monitor product demand, stock levels, and performance of product categories
Supplier Analysis	Identify key suppliers and optimize supply chain operations
Revenue Analysis	Generate revenue trends and identify high-performing regions or categories
Customer Satisfaction	Use review data to enhance product offerings and improve customer experience

1.4 Technical Stack

- Database Engine: MySQL 8.0+
 - Query Tool: MySQL Workbench
 - Concepts Applied: DDL, DML, TCL, Normalization (3NF), ACID Transactions
 - Dataset: 6 CSV files — Customers, Products, Suppliers, Orders, Order_Details, Reviews
-

2. Data Modeling & ER Diagram

2.1 Entity-Relationship Overview (Task 1 & 2)

The database is designed around six core entities. Each table is linked through well-defined primary and foreign key relationships to ensure referential integrity and eliminate data redundancy.

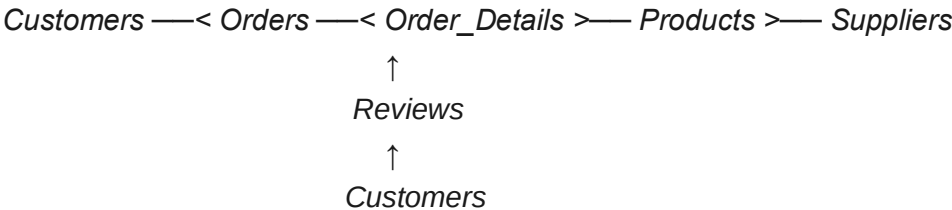
Table	Primary Key	Foreign Keys	Relationship
Customers	CustomerID	—	One-to-Many with Orders, Reviews
Suppliers	SupplierID	—	One-to-Many with Products
Products	ProductID	SupplierID	One-to-Many with Order_Details, Reviews
Orders	OrderID	CustomerID	One-to-Many with Order_Details
Order_Details	OrderDetailID	OrderID, ProductID	Resolves Many-to-Many between Orders & Products
Reviews	ReviewID	ProductID, CustomerID	Connects Customers to Products via feedback

2.2 Key Relationships

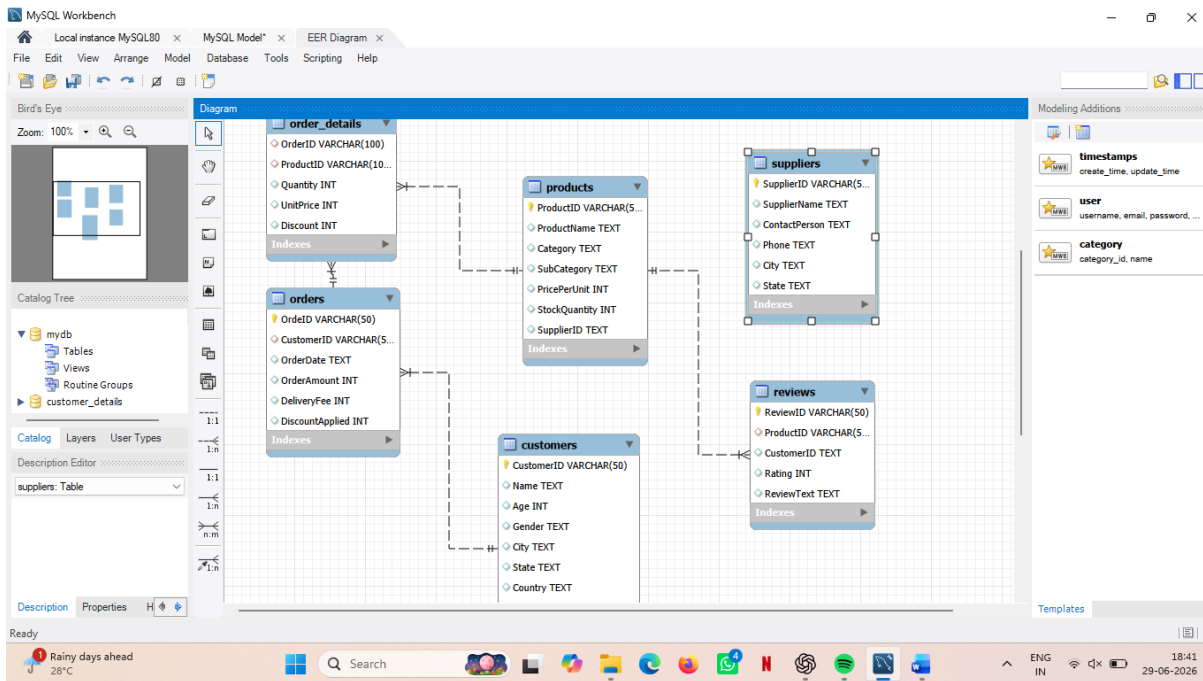
- A Customer can place many Orders (1:N — Customers → Orders)
- An Order can contain many Products via Order_Details (M:N resolved by Order_Details)
- A Supplier can supply many Products (1:N — Suppliers → Products)
- A Customer can write many Reviews, and a Product can receive many Reviews (1:N both ways)

2.3 ER Diagram Description

The ER diagram illustrates the following structure:



The bridge table Order_Details resolves the many-to-many relationship between Orders and Products, storing line-item details such as quantity, unit price, and discount per item.



3. Database Definition Language (DDL)

3.1 Database Setup

The database was created in MySQL Workbench using the following setup command:

```
Create database customer_details;
use customer_details;
```

3.2 Table Creation with Constraints (Task 4)

All six tables were created with appropriate data types, primary keys, foreign keys, NOT NULL constraints, UNIQUE constraints, CHECK constraints, and DEFAULT values to enforce data integrity at the schema level.

Customers Table — Key Constraints Applied:

```
CREATE TABLE Customers (
  CustomerID VARCHAR(50) PRIMARY KEY,
  Name VARCHAR(100) NOT NULL UNIQUE,
  Age INT NOT NULL CHECK (Age > 18),
  Gender VARCHAR(10),
  City VARCHAR(100),
  State VARCHAR(100),
  Country VARCHAR(100),
  SignupDate DATE,
  PrimeMember VARCHAR(3) DEFAULT 'No'
);
```

Constraint Type	Column	Rule
PRIMARY KEY	CustomerID	Uniquely identifies each customer
NOT NULL + CHECK	Age	Age must be present and > 18
UNIQUE	Name	No two customers can share the same name
DEFAULT	PrimeMember	Defaults to 'No' if not specified
FOREIGN KEY	SupplierID (Products)	Must exist in Suppliers table
CHECK	Rating (Reviews)	Rating must be between 1 and 5

4. Basic SELECT Queries (Task 3)

4.1 Retrieve Customers from a Specific City

This query filters the Customers table to return all records where the city matches a given value — useful for geographic customer segmentation.

```
select name,city from customer_details customers;
```

4.2 Fetch Products by Category

This query retrieves all products listed under the 'Fruits' category, helping product managers monitor category-level inventory.

```
select * from customer_details.products  
where Category like "Fruits";
```

These foundational queries demonstrate the use of the WHERE clause to filter rows based on column values — the most fundamental operation in SQL data retrieval.

5. Data Manipulation Language (DML)

5.1 INSERT — Adding New Products (Task 5)

Three new product records were inserted into the Products table to simulate new product onboarding:

```
insert into customer_details.products(ProductID, ProductName, Category, SubCategory,
PricePerUnit, StockQuantity, SupplierID)
value
(101, 'Laptop', 'Electronics', 'Electronics1',55000, 15,'756yht756yth'),
(102, 'Smartphone', 'Electronics','Electronics1', 25000, 30,'868uy8776u67'),
(103, 'Office Chair', 'Furniture','furniture1', 7500, 20,'35tr7tjhg7849');
```

5.2 UPDATE — Modifying Stock Quantity (Task 6)

The stock quantity for a specific product was updated to reflect a restocking event:

```
UPDATE customer_details.products
SET StockQuantity = 35
WHERE ProductID = '101';
```

5.3 DELETE — Removing a Supplier by City (Task 7)

A supplier was deleted based on their city, simulating the removal of a regional supplier from the system:

```
Delete from customer_details.suppliers
where city = 'Millerburgh';
```

DML operations allow for dynamic management of data. INSERT adds new records, UPDATE modifies existing data, and DELETE removes stale or irrelevant entries — all critical for maintaining an up-to-date database.

6. SQL Constraints (Task 8)

Beyond the constraints defined at table-creation time, additional constraints were applied via ALTER TABLE statements to reinforce data validation rules.

6.1 CHECK Constraint on Reviews.Rating *

Ensures that no rating below 1 or above 5 can be stored in the Reviews table:

```
alter table customer_details.Reviews  
add constraint chk_rating check (Rating between 1 and 5);
```

6.2 DEFAULT Constraint on Customers.PrimeMember

Ensures that when a new customer is added without specifying Prime membership status, it defaults to 'No':

```
alter table customer_details.customers  
modify PrimeMember varchar(10) default 'No';
```

These constraints act as business rules enforced at the database level, preventing invalid data from entering the system regardless of the application layer.

7. Clauses and Aggregations (Task 9)

7.1 WHERE Clause — Orders After January 1, 2024

Filters orders to only those placed after the specified date, supporting time-based reporting:

```
select * from customer_details.orders  
where Orderdate>2024-01-01;
```

7.2 HAVING Clause — Products with Average Rating > 4

Groups reviews by product and filters to only those with a high average rating — identifying top-rated products:

```
select p.ProductID,p.ProductName,SUM(s.Quantity) as TotalunitSales ,sum(s.Quantity*  
s.UnitPrice) as totalprice  
from customer_details.order_details as s  
join customer_details.Products as p  
on p.ProductID = s.ProductID  
group by p.ProductID, p.ProductName  
order by TotalunitSales DESC;
```

7.3 GROUP BY & ORDER BY — Rank Products by Total Sales

Aggregates order-level data to rank products by total units sold and revenue generated:

```
select p.ProductID,p.ProductName,SUM(s.Quantity) as TotalunitSales ,sum(s.Quantity* s.UnitPrice) as totalprice
from customer_details.order_details as s
join customer_details.Products as p
on p.ProductID = s.ProductID
group by p.ProductID, p.ProductName
order by TotalunitSales DESC;
```

8. Identifying High-Value Customers (Task 10)

Amazon Fresh wants to identify its most valuable customers based on total order spending. This was achieved in three progressive steps.

8.1 Total Spending Per Customer with Rank

A window function (RANK) was used to rank customers by their cumulative spending:

```
select c.CustomerID,c.Name,c.City,SUM(o.OrderAmount) as TotalSpending,RANK() over (order by
SUM(o.OrderAmount) desc) as SpendingRank
from customer_details.Customers as c
join customer_details.Orders as o on c.CustomerID = o.CustomerID
group by c.CustomerID, c.Name, c.City
order by SpendingRank;
```

8.2 Filter Customers Spending More Than ₹5,000

The HAVING clause filters results to return only high-value customers:

```
select c.CustomerID,c.Name,c.City,SUM(o.OrderAmount) as TotalSpending from
customer_details.Customers c
join customer_details.Orders as o on c.CustomerID = o.CustomerID
group by c.CustomerID, c.Name, c.City
having SUM(o.OrderAmount) > 5000
order by TotalSpending desc;
```

Business Impact: These customers can be targeted with loyalty programs, exclusive offers, or early access to new products to increase retention and lifetime value.

9.1 Total Revenue Per Order (Orders JOIN Order_Details)

Joins three tables to compute the total revenue and discount amount for each order:

```
select o.OrderID,o.OrderDate,SUM(od.Quantity*od.UnitPrice) as Total_Revenue from
customer_details.orders as o

join customer_details.order_details as od on o.OrderID=od.OrderID

group by o.OrderID,o.OrderDate

order by Total_Revenue desc ;
```

9.2 Most Active Customers in 2025

Identifies the top 10 customers who placed the most orders within the year 2025:

```
select c.CustomerID,c.Name,count(o.OrderID) as TotalOrders  from customer_details.customers as
c

join customer_details.orders as o

on c.CustomerID = o.CustomerID

where o.OrderDate between '2025-01-01' and '2025-03-31'

group by c.CustomerID, c.Name

order by TotalOrders desc limit 10;
```

9.3 Supplier with the Most Products in Stock

```
select  s.SupplierID,s.SupplierName,SUM(p.StockQuantity) as TotalStock,COUNT(p.ProductID) as
ProductCount from  customer_details.suppliers as s

join customer_details.Products as p on s.SupplierID = p.SupplierID

group by s.SupplierID, s.SupplierName

order by TotalStock desc ;
```

10. Normalization to 3NF (Task 12)

The original Products table stored Category and SubCategory as plain text strings, leading to data redundancy (same category name repeated across many rows) and update anomalies. To resolve this, the table was normalized to Third Normal Form (3NF).

10.1 Problem with the Original Design

- Repeating category names wastes storage space
- A single spelling error would corrupt all associated product lookups
- Renaming a category requires updating hundreds of row

10.2 Solution — Separate Lookup Tables

```
-- Create Categories lookup table
CREATE TABLE IF NOT EXISTS Categories (
    CategoryID INT AUTO_INCREMENT PRIMARY KEY,
    CategoryName VARCHAR(100) NOT NULL UNIQUE
);

-- Create SubCategories lookup table
CREATE TABLE IF NOT EXISTS SubCategories (
    SubCategoryID INT AUTO_INCREMENT PRIMARY KEY,
    SubCategoryName VARCHAR(100) NOT NULL,
    CategoryID INT,
    FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);
```

10.3 Migration — Populate & Link

```
-- Populate Categories from existing Products data
INSERT INTO Categories (CategoryName)
SELECT DISTINCT Category FROM Products WHERE Category IS NOT NULL;

-- Add FK columns to Products
ALTER TABLE Products
    ADD COLUMN CategoryID INT,
    ADD COLUMN SubCategoryID INT,
    ADD CONSTRAINT fk_cat FOREIGN KEY (CategoryID) REFERENCES
Categories(CategoryID),
    ADD CONSTRAINT fk_subcat FOREIGN KEY (SubCategoryID) REFERENCES
SubCategories(SubCategoryID);

-- Update the new FK columns
UPDATE Products p
JOIN Categories c ON p.Category = c.CategoryName
JOIN SubCategories sc ON p.SubCategory = sc.SubCategoryName
AND sc.CategoryID = c.CategoryID
SET p.CategoryID = c.CategoryID,
    p.SubCategoryID = sc.SubCategoryID
```

11. Subqueries and Nested Queries (Task 13)

11.1 Top 3 Products by Sales Revenue

A derived table (inline view) was used to first compute total revenue per product, then the outer query selects the top 3:

```
select ProductID, ProductName, TotalRevenue from (select
p.ProductID,p.ProductName,SUM(od.Quantity * od.UnitPrice) as TotalRevenue
from customer_details.Products as p
join customer_details.Order_Details as od on p.ProductID = od.ProductID
group by p.ProductID, p.ProductName) as ProductRevenue
order by TotalRevenue desc limit 3;
```

11.2 Customers Who Have Not Placed Any Orders

A NOT IN subquery identifies customers in the Customers table who have no corresponding entries in the Orders table:

```
select CustomerID, Name, City, SignupDate from customer_details.Customers
where CustomerID NOT IN (select distinct CustomerID from customer_details.Orders where
CustomerID is not null);
```

Business Impact: These inactive customers represent a re-engagement opportunity. Targeted email campaigns or first-order discounts can be offered to convert registered-but-inactive users into buyers.

12. Real-World Actionable Insights (Task 14)

12.1 Cities with the Highest Concentration of Prime Members

This query ranks cities by the count and percentage of Prime members to guide regional marketing spend:

```
SELECT City,
COUNT(*) AS TotalCustomers,
SUM(CASE WHEN PrimeMember = 'Yes' THEN 1 ELSE 0 END) AS PrimeMembers,
ROUND(
SUM(CASE WHEN PrimeMember = 'Yes' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2
) AS PrimeMemberPct
FROM Customers
GROUP BY City
HAVING PrimeMembers > 0
ORDER BY PrimeMembers DESC
LIMIT 10;
```

12.2 Top 3 Most Frequently Ordered Categories

Identifies the product categories that appear most often across all order line items:

```
select p.Category, COUNT(od.OrderID) as TimesOrdered, SUM(od.Quantity) as TotalUnitsSold from
customer_details Order_Details as od

join customer_details.Products p on od.ProductID = p.ProductID

group by p.Category

order by TimesOrdered desc limit 3;
```

Business Impact: These insights allow Amazon Fresh to prioritize stock replenishment for high-demand categories, adjust marketing budgets toward high-Prime cities, and tailor promotional campaigns to customer behavior.

13. ACID Transaction — TCL Commands (Bonus)

A real-world transaction scenario was implemented: a customer places an order, the order and its line items are recorded, and the product stock is decremented — all atomically using START TRANSACTION, COMMIT, and ROLLBACK.

13.1 Transaction Implementation

```
START TRANSACTION;

-- Step 1: Insert a new order
INSERT INTO Orders (OrderID, CustomerID, OrderDate, OrderAmount, DeliveryFee,
DiscountApplied)
VALUES ('ORD-TXN-001', '4331c5fa-0c04-4383-82b1-e30894c79faa', CURDATE(), 1500.00, 50.00,
0);

-- Step 2: Insert order detail
INSERT INTO Order_Details (OrderID, ProductID, Quantity, UnitPrice, Discount)
VALUES ('ORD-TXN-001', '2aa28375-c563-41b5-aa33-8e2c2e0f4db9', 2, 207.00, 0);

-- Step 3: Deduct stock (with safety check to prevent negative stock)
UPDATE Products
SET StockQuantity = StockQuantity - 2
WHERE ProductID = '2aa28375-c563-41b5-aa33-8e2c2e0f4db9'
AND StockQuantity >= 2;

COMMIT; -- All steps succeeded — persist changes
-- ROLLBACK; -- Uncomment to undo all steps if any step fails
```

13.2 ACID Properties Applied

Property	Principle	How It's Applied
Atomicity	All or Nothing	START TRANSACTION ensures all 3 steps succeed or none persist
Consistency	Valid State Always	Constraints (FK, CHECK) prevent partial or invalid data
Isolation	Concurrent Safety	Transaction is isolated from other concurrent operations
Durability	Permanent on Commit	COMMIT makes changes permanent; ROLLBACK reverts on failure

14. Project Summary and Outcomes

14.1 What Was Accomplished

Task	Status
ER Diagram & Relationship Mapping (Task 1 & 2)	Completed
Basic SELECT Queries — City & Category Filter (Task 3)	Completed
DDL — Customers Table with Constraints (Task 4)	Completed
DML — INSERT 3 Products (Task 5)	Completed
DML — UPDATE Stock Quantity (Task 6)	Completed
DML — DELETE Supplier by City (Task 7)	Completed
SQL Constraints — CHECK & DEFAULT (Task 8)	Completed
Clauses — WHERE, HAVING, GROUP BY (Task 9)	Completed
High-Value Customer Identification (Task 10)	Completed
Complex Joins & Aggregations (Task 11)	Completed
Normalization to 3NF (Task 12)	Completed
Subqueries & Nested Queries (Task 13)	Completed
Real-World Actionable Insights (Task 14)	Completed
ACID Transaction with TCL — Bonus	Completed

14.2 Key Learnings

- Designing a normalized relational schema reduces redundancy and improves data integrity
- Window functions like RANK() provide powerful ranking capabilities without subqueries
- ACID transactions are essential for multi-step database operations in production systems
- Normalization (3NF) separates concerns between entity attributes and lookup values
- Aggregation functions combined with GROUP BY and HAVING unlock deep analytical insights
- Subqueries and derived tables allow complex business questions to be answered in a single SQL statement

14.3 Technical Tags

*SQL • DDL • DML • TCL • ACID TRANSACTIONS • DATABASE NORMALIZATION • E-COMMERCE
ANALYTICS • BUSINESS INSIGHTS • MYSQL WORKBENCH*